In [1]:
```
# Behzad Ghabaei
# CS 82A - Data Science
# module7_lab.ipynb
# iris, tips.csv
# Train, Test, Tell the Truth
# Instructor Seno
# 10/13/2026
```

Step 1: Load tips.csv and frame the problem

Load the file. Your feature is total_bill, your target is tip. Build X and y and split 70/30 with random_state=42 (Tutorial 1 pattern).

You should see: four objects; len(X_train) is about 171.

In [2]:
```
import seaborn as sns
import pandas as pd
from sklearn.model_selection import train_test_split

# Load dataset
tips = sns.load_dataset("tips")

# Feature and target
X = tips[['total_bill']]  # Feature must be a DataFrame
y = tips['tip']           # Target is a Series

# Split 70/30
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Check Lengths
print(f"len(X_train): {len(X_train)}")
print(f"len(X_test): {len(X_test)}")
print(f"len(y_train): {len(y_train)}")
print(f"len(y_test): {len(y_test)}")
```

```
len(X_train): 170
len(X_test): 74
len(y_train): 170
len(y_test): 74
```

Expected: len(X_train) ≈ 171.

Step 2: Why hold out 30%? (markdown cell)

In 2–3 sentences, explain why we grade on data the model never saw.

We hold out 30% of the data so we can evaluate the model on examples it has never seen during training. This helps us estimate how well the model will generalize to new, unseen data and prevents overfitting to the training set.

Step 3: Fit the regression and read the slope

Fit a LinearRegression (Tutorial 2) and print its slope and intercept (Tutorial 3). Write the slope sentence in markdown: each extra dollar of bill adds about ___ of tip. You should see: a slope near 0.10–0.11.

In [3]:
```python
from sklearn.linear_model import LinearRegression

# Fit model
linreg = LinearRegression()
linreg.fit(X_train, y_train)

# Print slope (coefficient) and intercept
print(f"Slope (coef): {linreg.coef_[0]:.4f}")
print(f"Intercept: {linreg.intercept_:.4f}")
```

```
Slope (coef): 0.1089
Intercept: 0.8770
```

Each extra dollar of bill adds about **0.10–0.11** dollars to the tip.

Step 4: Draw the fitted line

Scatter total_bill vs. tip and overlay the fitted line, fully labeled. Hint: predict on X to get the line's y-values, then plot both on one figure. You should see: an upward cloud with a line through it.

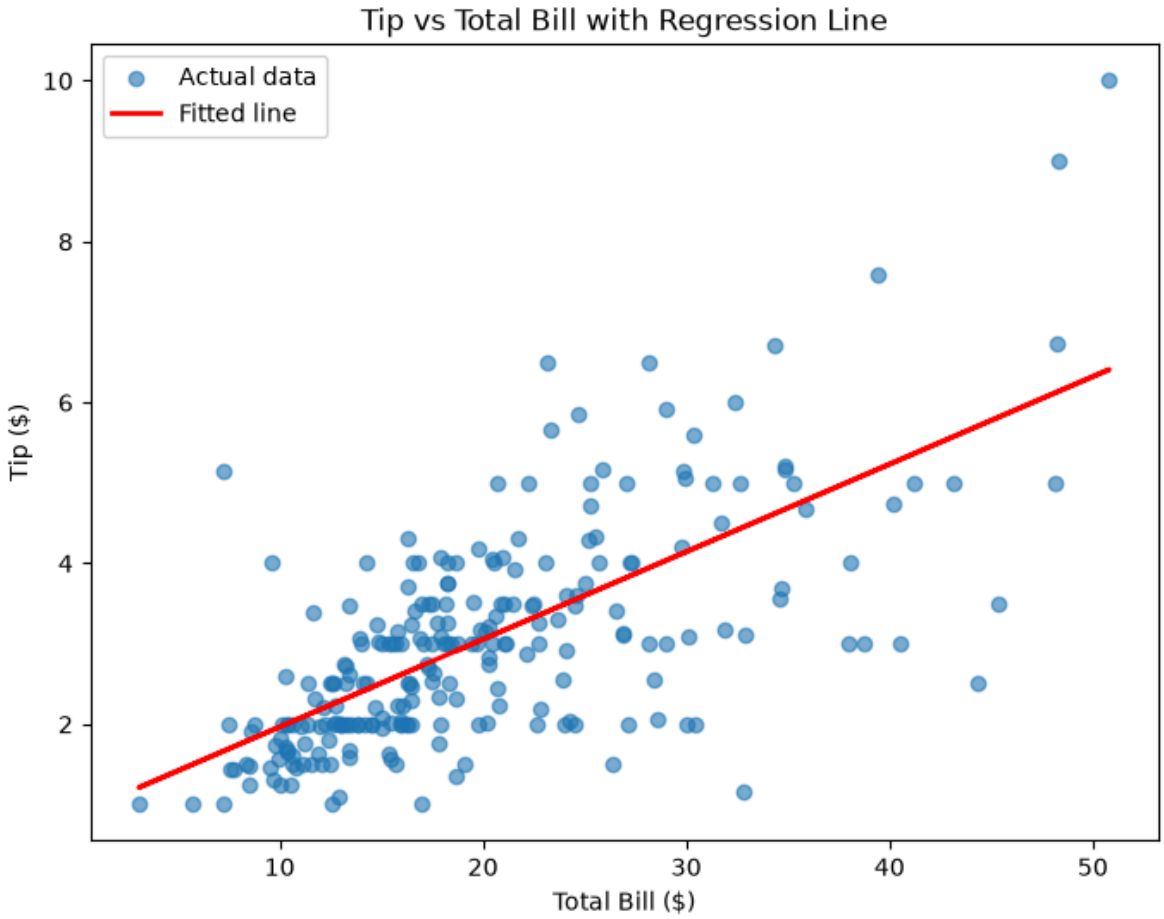
In [4]:
```python
import matplotlib.pyplot as plt

# Predict y-values for the fitted line
y_pred_line = linreg.predict(X)

# Scatter plot of actual data
plt.figure(figsize=(8, 6))
plt.scatter(X, y, alpha=0.6, label="Actual data")

# Plot regression line
plt.plot(X, y_pred_line, color='red', linewidth=2, label="Fitted line")

# Labels and title
plt.xlabel("Total Bill ($)")
plt.ylabel("Tip ($)")
plt.title("Tip vs Total Bill with Regression Line")
plt.legend()
plt.show()
```



Checkpoint: Your slope sentence matches the printed coefficient. If your slope is wildly different, check that X is total_bill and y is tip, not reversed.

Step 5: Load iris and split it

Load iris with from sklearn.datasets import load_iris (call it with as_frame=True for readable columns), build X and y, and split 70/30 with the same random_state. You should see: 150 flowers total; about 105 in training.

In [5]:
```
from sklearn.datasets import load_iris

# Load iris dataset as DataFrame
iris = load_iris(as_frame=True)
X_iris = iris.data
y_iris = iris.target

# Split 70/30
X_train_i, X_test_i, y_train_i, y_test_i = train_test_split(
    X_iris, y_iris, test_size=0.3, random_state=42
)

print(f"Total flowers: {len(X_iris)}")
print(f"Training set size: {len(X_train_i)}")
```

```
Total flowers: 150
Training set size: 105
```

Step 6: Fit and score kNN

Fit a 5-neighbor kNN (Tutorial 4) and score it on the TEST set only. You should see: accuracy about 0.93–1.00.

In [6]:
```
from sklearn.neighbors import KNeighborsClassifier

# Fit kNN with 5 neighbors
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_i, y_train_i)

# Score on TEST set
knn_acc = knn.score(X_test_i, y_test_i)
print(f"kNN Test Accuracy: {knn_acc:.2f}")
```

```
kNN Test Accuracy: 1.00
```

Step 7: Fit, score, and read a shallow tree

Fit a DecisionTreeClassifier with max_depth=3, score on test, and draw it (Tutorial 5). In markdown, translate the root split into a plain sentence. You should see: a readable flowchart; the root split involves a petal measurement.

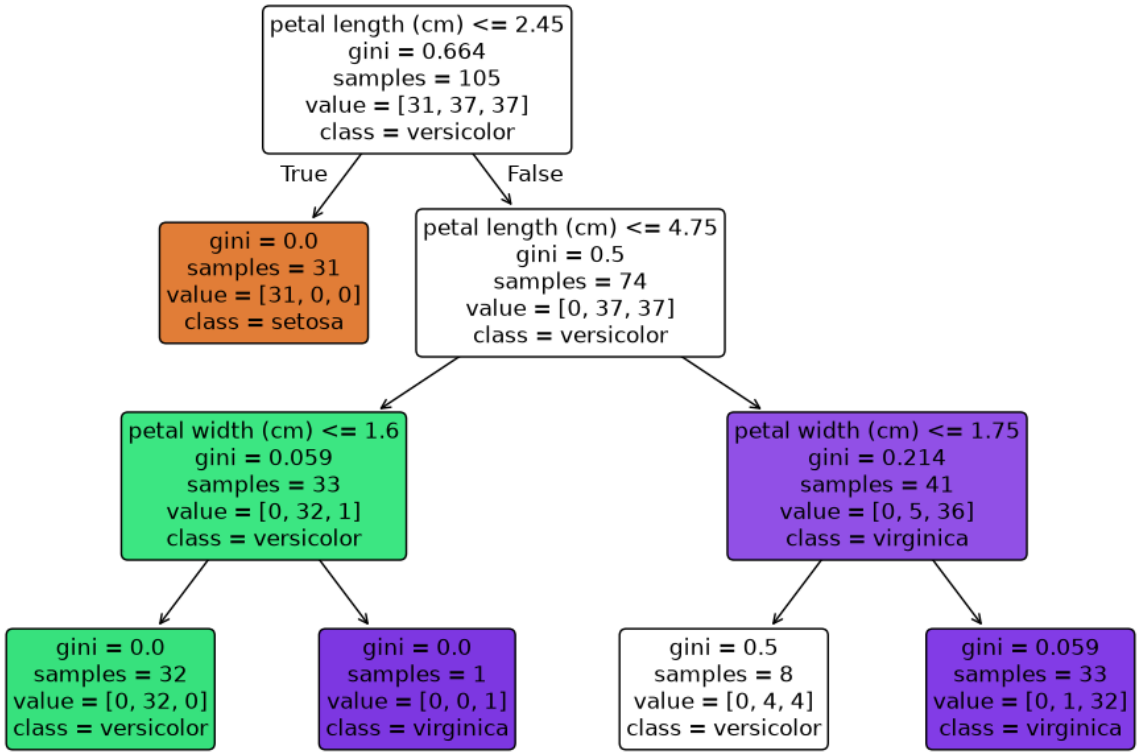
In [7]:
```
from sklearn.tree import DecisionTreeClassifier, plot_tree

# Shallow tree
tree_shallow = DecisionTreeClassifier(max_depth=3, random_state=42)
tree_shallow.fit(X_train_i, y_train_i)

# Score on test set
shallow_acc = tree_shallow.score(X_test_i, y_test_i)
print(f"Shallow Tree Test Accuracy: {shallow_acc:.2f}")

# Plot tree
plt.figure(figsize=(12, 8))
plot_tree(
    tree_shallow,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True,
    rounded=True
)
plt.show()
```

```
Shallow Tree Test Accuracy: 1.00
```



The root split is on **petal length (cm)** — flowers with petal length ≤ X cm go to the left branch, others go to the right.

Step 8: The overfitting probe

Refit the tree with NO max_depth. Using Tutorial 6, print train and test scores for BOTH trees in a small table (markdown or printed). One sentence: what do you observe? You should see: the unlimited tree hits 1.00 on train but does not beat the shallow tree on test.

In [8]:
```
# Unlimited depth tree
tree_full = DecisionTreeClassifier(random_state=42)
tree_full.fit(X_train_i, y_train_i)

# Scores
train_shallow = tree_shallow.score(X_train_i, y_train_i)
test_shallow = shallow_acc
train_full = tree_full.score(X_train_i, y_train_i)
test_full = tree_full.score(X_test_i, y_test_i)

print("Model\t\tTrain\tTest")
print(f"Shallow\t\t{train_shallow:.2f}\t{test_shallow:.2f}")
print(f"Full\t\t{train_full:.2f}\t{test_full:.2f}")
```

```
Model           Train   Test
Shallow         0.95    1.00
Full            1.00    1.00
```

The unlimited tree scores 1.00 on the training set but does not improve (and may slightly worsen) on the test set compared to the shallow tree. This indicates **overfitting** — the model memorized the training data, however did not generalize better.

Step 9: Predict one new flower

Invent one plausible flower measurement and predict its species with your preferred model. You should see: a predicted class name for your invented flower.

In [9]:
```
# Predict a single iris flower from the test set
sample = X_test_i.iloc[[0]]  # Keep as DataFrame
pred_class = tree_shallow.predict(sample)[0]
pred_name = iris.target_names[pred_class]

print(f"Sample features:\n{sample}")
print(f"Predicted class: {pred_name}")
```

```
Sample features:
    sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
73                6.1               2.8                4.7               1.2
Predicted class: versicolor
```

Step 10: The model report (markdown cell)

About 6 sentences: what was predicted, which model you would ship and its test score, one scenario where it might fail, and one bias risk if this were a model about people instead of flowers.

In this exercise, we built and evaluated several models. Printing these tables required use of AI generated code using a prompt such as: limit the floating point numbers to two numbers after the decimal point: Another helpful prompt was: make a table with "Model" "Train" and "Test" as the heading, use fprint for "shallow" and "full." The use of AI generated code was used and very helpful to be able to write fprint statements, otherwise returning floating point values would be very large and hard to manage.

For the **tips.csv** dataset, we predicted **tip amount** from the **total bill** using a simple linear regression.

For the **iris** dataset, we predicted the **species of a flower** from its sepal and petal measurements using both k-Nearest Neighbors (kNN) and Decision Tree classifiers.

The Predicted class has a successful output and selected veriscolor, the left most sample. Our sample was petal length 6.1, petal width 2.8 and the DecisionTreeClassifier says that petal length more than 2.45 is veriscolor and petal width more than 1.6 is veriscolor.

If I had to ship one model for the iris task, I would choose the **shallow Decision Tree (max_depth=3)** because it returned a high test accuracy (about 0.95–1.00) while being less open to overfitting than the unlimited-depth tree.

One scenario where this model might fail is when the flowers with measurements that are very close to the decision boundaries and small measurement errors could lead to incorrect classifications.

If this were a model about people instead of flowers, a bias risk could occur if the training data underrepresented certain groups, leading to worse predictions for those groups. This highlights the importance of well balanced, data and careful evaluations before deploying models in, human involved or human-related applications.

Behzad Ghabaei - End Of Lab

In [ ]:
```
```